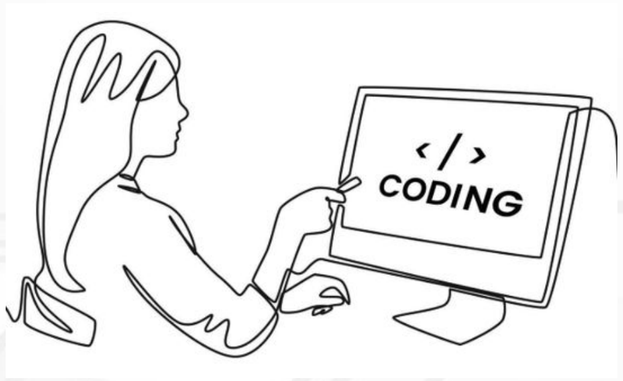
FADENA
FACULTAD DE DEFENSA NACIONAL

Lenguajes de Programación

Clase 2: Pseudocódigo – parte 1

Docentes:

- Ing. Sebastián Balestrini
- Esp. Ing. Gustavo Di Stefano



Objetivos

- Aprender a escribir algoritmos en pseudocódigo.
- Aplicar estructuras de control (decisión y repetición).
- Consolidar la lógica algorítmica antes de programar en C.



Herramientas

- Diseño y ejecución paso a paso un diagrama de flujo para ver cómo fluyen los datos y entender la lógica. https://flowrun.io

- Para mejor legibilidad del pseudocódigo se puede usar Notepad++ (o similares) utilizando el formato del lenguaje C. https://notepad-plus-plus.org/downloads

Pseudocódigo

Definición breve: Es una forma de escribir un algoritmo con pasos claros, sin usar la sintaxis de un lenguaje de programación.

¿**Para qué sirve**?

- Planificar el código
- Entender la lógica
- Comunicar ideas claras

No se ejecuta, se piensa.

El pseudocódigo entrena el pensamiento lógico.

Sintaxis básica:

Acción	Pseudocódigo	D. de Flujo
Comienzo	Inicio	
Ingreso	Leer A	
Salida	Escribir A	
Decisión	Si	
Asignacion	S = A +B	
Fin del algoritmo	Fin	

Pseudocódigo

Ejemplo 1: Calcular el promedio de tres números ingresados por consola

```
1  inicio()
2  {
3      imprimir("Ingrese 3 números: ");
4      ingresar(a,b,c);
5      promedio = (a + b + c)/3;
6      imprimir("El promedio es: ", promedio);
7  }
8
```

Salida (consola)

Ingrese 3 números:

(Si a = 2, b = 5, c = 2)

El promedio es: 3

Pseudocódigo

Ejemplo 2: hallar el área de un rectángulo ingresando sus lados

```
1  inicio()
2  {
3      imprimir("Ingrese lados del rectangulo: ");
4      ingresar(a,b);
5      area = a * b;
6      imprimir("El area es: ", area);
7  }
```



Salida (consola)

Ingrese lados del rectangulo

(Si a = 5, b = 2)

El area es: 10

FADENA
FACULTAD DE DEFENSA NACIONAL

Pseudocódigo: Variables y Constantes

Variable:

- Tiene nombre, tipo y valor modificable.
- Representa una posición de memoria donde se almacena un dato.
- El valor se asigna más adelante.

Constante:

- Tiene nombre, tipo y valor inalterable.
- El valor se define al crearla y no cambia durante la ejecución.

Ejemplo 3: hallar el área de un circulo

```
definir PI 3.1416
inicio()
{
    imprimir("Ingrese radio: ");
    ingresar(r);
    area = PI * r * r;
    imprimir("El area es: ", area);
}
```

Pseudocódigo: Estructuras de control

¿Para que sirven?

Permiten controlar el flujo del programa:

- Tomar decisiones
- Repetir acciones
- Modificar el orden normal de ejecución

Tipos de estructuras de control

- **Secuencial** (vista anteriormente)
- **Alternativa o de decisión** (vemos ahora)
- **Repetición** (más adelante)

Ejemplo 4: Mostrar el área solo si el radio es mayor que 0

```
definir PI 3.1416
inicio()
{
    imprimir("Ingrese radio: ");
    ingresar(r);
    si (r > 0)
    {
            area = PI * r * r;
            imprimir("El area es: ", area);
    }
}
```

Pseudocódigo: Decisiones anidadas

¿Que son?

- Una estructura de decisión anidada ocurre cuando dentro de un bloque si colocamos otro si.
- Se usa cuando hay más de dos alternativas posibles.

Buenas prácticas

- Usar indentación clara para mejorar la lectura.
- Las llaves {} son necesarias cuando hay más de una instrucción en un bloque.
- Se pueden escribir de diferentes formas según la lógica que queramos priorizar.

Ejemplo 5: Comparar dos números e identificar si uno es mayor o si son iguales.

```
1  inicio()
2  {
3    ingresar(a, b);
4    si (a > b)
5      imprimir("El número ", a, " es el mayor");
6    sino
7      si (a == b)
8        imprimir("Son iguales");
9      sino
10       imprimir("El número ", b, " es el mayor");
11 }
```

Pseudocódigo: Decisiones anidadas

Ejemplo 6: Clasificación de una nota

Este algoritmo evalúa una nota ingresada del 1 al 10 y la clasifica según el siguiente criterio:

- Menor a 6 → Desaprobado
- Entre 6 y 8 → Aprobado
- 9 o 10 → Sobresaliente
- Fuera del rango → Error

Usamos decisiones anidadas para ir filtrando cada condición en orden, de forma clara y estructurada.

```
inicio()
{
  ingresar(nota);
  si (nota >= 1 y nota <= 10)
  {
    si (nota < 6)
      imprimir("Desaprobado");
    sino
      si (nota < 9)
        imprimir("Aprobado");
      sino
        imprimir("Sobresaliente");
  }
  sino
    imprimir("Error: nota fuera de rango");
}
```

Pseudocódigo: Estructura de control múltiple

¿Que es?

- La estructura de control múltiple nos permite elegir entre muchas alternativas de forma más clara y eficiente que los si anidados.
- Es ideal para casos como: selección de meses, menús de opciones o cálculos según operadores.

¿Por qué usarla?

- Evita la sobrecarga de si – sino encadenados
- Mejora la claridad del código
- Facilita la lectura y el mantenimiento

Sintaxis básica

```
según (expresión)
{
  caso valor1:
    acciones;
    salir;
  ...
  caso valorN:
    acciones;
    salir;
  defecto:
    acciones por defecto;
    salir;
}
```

Pseudocódigo: Estructura de control múltiple (switch case)

Ejemplo 7: Ingresar dos números y un operador aritmético para realizar el cálculo.



```
inicio()
{
    ingresar(a, b);
    ingresar(op);  // puede ser '+', '-', '*', '/'

    según (op)
    {
        caso '+':
            r = a + b;
            salir;
        caso '-':
            r = a - b;
            salir;
        caso '*':
            r = a * b;
            salir;
        caso '/':
            si (b == 0)
                r = 0;
            sino
                r = a / b;
            salir;
        defecto:
            imprimir("Operador inválido");
            salir;
    }

    imprimir("El resultado es: ", r);
}

```

Pseudocódigo: Operadores relacionales y Módulo

Operadores de relación

Operador	Significado
>	Mayor que
<	Menor que
>=	Mayor o igual que
<=	Menor o igual que
==	Igual
!=	Distinto

Operador Módulo %

- Calcula el resto de una división.
- Se usa para saber si un número es par, impar o múltiplo.

Ejemplo 8: dado un número calcular si es multiplo de 5

```
1  inicio()
2  {
3      ingresar(n);
4      si (n % 5 == 0)
5          imprimir("Es múltiplo de 5");
6      sino
7          imprimir("No es múltiplo de 5");
8  }
9
```

Pseudocódigo: Operadores lógicos

Operadores lógicos

Operador	Significado
&& (AND)	verdadero si ambas condiciones lo son
|| (OR)	Verdadero si al menos una lo es.
! (NOT)	Invierte el valor lógico.

Ejemplo 9: Control de encendido de estufa con operadores lógicos

```
1  inicio()
2  {
3      ingresar(temperatura);
4      ingresar(ventanas_abiertas);  // verdadero o falso
5
6      si (temperatura < 15 && !ventanas_abiertas)
7          imprimir("Encender estufa");
8      sino
9          si (temperatura >= 24 || ventanas_abiertas)
10             imprimir("Apagar estufa");
11         sino
12             imprimir("Mantener estado actual");
13 }
```

Que aprendimos hasta acá ?

- Qué es el pseudocódigo y para qué sirve
- Cómo se declaran variables y constantes
- Las estructuras de control:
    - Secuencial
    - Decisión simple, doble y anidada
    - Control múltiple (switch case)
- Uso de operadores relacionales (>, <, ==, !=, etc.)
- El operador módulo % para verificar múltiplos o paridad
- Los operadores lógicos:
    - && Y lógico
    - || O lógico
    - ! Negación



Próximo paso

Prácticas con pseudocódigo y luego terminamos de presentar las estructuras de control con pseudocódigo

¿Preguntas o dudas?



FADENA
FACULTAD DE DEFENSA NACIONAL